

Metta-stasizing SUMO: A Typed MeTTa Encoding for Type Checking, PLN Reasoning, and Hyperseed Alignment

Abstract

This document proposes a practical, formally checkable way to port SUMO (Standard Upper Merged Ontology) into MeTTa (the Hyperon / AtomSpace language), while adding an explicit, structured higher-order type system.

The design is guided by three goals:

- 1) Make MeTTa-SUMO easy to type-check and sanity-check (arity, domains/ranges, subtyping, disjointness, and higher-order meta-axioms).
- 2) Make it easy to run PLN-style reasoning on noisy semantic parses and structured datasets, using SUMO as type- and axiom-guidance rather than relying on LLM glue alone.
- 3) Provide an explicit bridge from SUMO-level meanings to Hyperseed concepts, so that everyday inference problems (text + data) can be routed into Hyperseed-style inference guidance without having to map semantic parses directly into hyperseed-math formalisms.

Examples are sketched for (i) AI ethics, (ii) crypto-finance, and (iii) biology (metabolic pathways and causal explanations for genomics).

Contents

1	High-level architecture	1
2	Representation choices	2
2.1	Two levels of meaning: object-level vs meta-level	2
2.2	Namespaces	2
3	A higher-order type system for MeTTa-SUMO	2
3.1	Core type universe	2
3.2	Relation signatures (domain/range as typing)	3
3.3	Subtyping and class-as-type discipline	3
3.4	Typed application rules	4
3.5	Context-indexed truth and evidence	4
3.6	Formal type-checking passes	4
4	Using SUMO for PLN reasoning on semantic parses + datasets	5
4.1	Why SUMO helps	5
4.2	PLN-friendly rule templates derived from SUMO	5
4.3	Combining evidence from text and data	6

5	Connecting SUMO to Hyperseed	6
5.1	What the SUMO $\leftrightarrow$ Hyperseed bridge should be	6
5.2	Representing mappings as graded correspondences	6
5.3	Using the bridge for inference guidance	7
6	Domain examples	7
6.1	AI ethics example: policy compliance and value conflicts	7
6.2	Crypto-finance example: on-chain pattern recognition + causal explanations	8
6.3	Biology example: metabolic pathways as pattern-flow networks	9
6.4	Biology example: causal explanations for genomics data	9
7	Suggested MVP experiments	10
8	Summary	11

1 High-level architecture

The basic idea is to build a *three-layer stack*:

- L1: Typed MeTTa-SUMO core.** Port SUMO KIF axioms into a MeTTa representation with explicit type signatures for classes, relations, functions, and meta-relations (e.g. `domain`, `subclass`).
- L2: Evidence and inference layer (PLN-friendly).** Represent semantic parsing outputs and dataset facts as evidence-bearing assertions (potentially uncertain, potentially conflicting). Use type constraints and SUMO axioms to guide inference and control search.
- L3: SUMO $\leftrightarrow$ Hyperseed bridge.** Represent a mapping from SUMO symbols and structures (classes, relations, patterns) into Hyperseed concepts and structures, using graded correspondences and explicit contexts. Use this bridge to: (a) interpret domain facts in a Hyperseed framing, (b) use Hyperseed concepts as inference guidance heuristics and explanations, and (c) translate back into SUMO-level statements when needed for application output.

2 Representation choices

2.1 Two levels of meaning: object-level vs meta-level

SUMO is mostly first-order but includes meta-predicates about predicates/classes. MeTTa can represent both, but it is helpful to explicitly separate:

- **Object-level statements** about individuals, e.g. `(parent John Mary)`.
- **Meta-level statements** about symbols, e.g. domain/range declarations or talking about a predicate as a first-class entity.

To support this, define two parallel encodings:

- (a) **Direct application form (fast, type-checkable):**
 $(P \ a_1 \ \dots \ a_n)$ where P is a relation symbol with a declared signature.

(b) **Reified form (meta-reasoning-friendly):**

(Holds P (Args $a_1 \dots a_n$)) where P is treated as a term.

A bridge rule links them (conceptually, not necessarily as an always-fired rewrite):

$$(P \ a_1 \ \dots \ a_n) \leftrightarrow \text{Holds}(P, \text{Args}(a_1, \dots, a_n)).$$

Keeping both encodings available makes it easier to do higher-order manipulation without sacrificing efficient type checking on ordinary facts.

2.2 Namespaces

In practice, use namespaces to avoid collisions:

- `SUMO::` for ported SUMO symbols.
- `HS::` for Hyperseed concepts/symbols.
- `DOM::` for domain-specific extensions (ethics/finance/bio).

3 A higher-order type system for MeTTa-SUMO

3.1 Core type universe

We want types rich enough to express: (i) classes and subclassing, (ii) relation signatures, (iii) meta-relations about relations, and (iv) contextual and graded truth.

A minimal starting point:

```
; Meta-types (kinds)
(: Type      Type)
(: Class     Type)
(: RelSym    Type)
(: FunSym    Type)
(: Prop      Type)
(: Context   Type)
(: Evidence  Type)

; Some useful ground types
(: Entity    Type)
(: Nat       Type)
```

Then define how SUMO-level symbols live in this universe:

```
; SUMO class symbols are of type Class
(: SUMO::Human      Class)
(: SUMO::Organization Class)

; SUMO relation symbols are of type RelSym
(: SUMO::parent     RelSym)
(: SUMO::agent      RelSym)
```

3.2 Relation signatures (domain/range as typing)

For each SUMO relation R , generate a *signature*:

```
; A signature object says what argument types a relation expects.
(: Sig Type)
(: mkSig (-> Nat (List Type) Type Sig))

; Example: parent : Human x Human -> Prop
(: sig:SUMO::parent Sig)
(= sig:SUMO::parent (mkSig 2 (List Entity Entity) Prop))
```

In real ports, you derive these signatures from SUMO domain/range axioms (and from conventions for functional terms).

3.3 Subtyping and class-as-type discipline

Treat SUMO subclassing as subtyping:

```
(: Subclass (-> Class Class Prop))
(: Subtype (-> Type Type Prop))

; Bridging rule (conceptual): a SUMO subclass edge induces a type-subtype edge.
; (Subclass A B) ==> (Subtype (TypeOf A) (TypeOf B))
```

There are multiple ways to implement `TypeOf`. A pragmatic approach: create a distinct type atom for each class:

```
(: TypeOf (-> Class Type))

(: T:Human      Type)
(: T:Organization Type)
(= (TypeOf SUMO::Human)      T:Human)
(= (TypeOf SUMO::Organization) T:Organization)
```

Then instances can be typed:

```
(: John T:Human)
(: Acme T:Organization)
```

3.4 Typed application rules

A type checker can implement the following judgments.

Term typing.

$$\Gamma \vdash a : T$$

meaning term a has type T in environment Γ .

Relation application. If R has signature $(T_1, \dots, T_n) \rightarrow \text{Prop}$ and each a_i has type T_i (or a subtype), then $(R \ a_1 \ \dots \ a_n)$ is well-typed.

Subtyping. Use standard subsumption:

$$\frac{\Gamma \vdash a : T \quad T \leq U}{\Gamma \vdash a : U}$$

3.5 Context-indexed truth and evidence

To support PLN-like and Hyperseed-like workflows, represent evidence explicitly.

A minimal pattern:

```
; Evidence object (can be PLN stv, p-bit, or a wrapper that holds both)
(: TV Type)
(: stv (-> Number Number TV))      ; (strength, confidence)
(: pbit (-> Number Number TV))      ; (pos, neg)

(: Assert (-> Context Prop TV Prop))
; Assert(C, P, tv) is itself a proposition in the KB, enabling meta-reasoning.
```

You can then store:

```
(: C:text Context)
(: C:data Context)

; Parsed from a sentence with uncertainty:
(Assert C:text (SUMO::parent John Mary) (stv 0.72 0.45))

; From a structured dataset with higher confidence:
(Assert C:data (SUMO::parent John Mary) (stv 0.95 0.90))
```

3.6 Formal type-checking passes

A realistic pipeline uses multiple passes:

- Pass 1: Symbol hygiene pass:** every symbol used is declared (or auto-declared into a quarantine namespace).
- Pass 2: Arity pass:** every (R ...) application matches the declared arity.
- Pass 3: Signature pass:** argument types satisfy relation signature types (up to subtyping).
- Pass 4: Meta-axiom pass:** meta-relations like `domain`, `range`, `subclass` are applied to appropriate meta-types (e.g. a `RelSym` where expected).
- Pass 5: Disjointness and contradiction pass:** identify contradictions such as: `Disjoint(A,B)` plus `x:A` and `x:B`. Decide whether these are treated as: (a) hard errors (strict mode), or (b) paraconsistent tensions (graded mode).

4 Using SUMO for PLN reasoning on semantic parses + datasets

4.1 Why SUMO helps

Semantic parsing from text tends to yield:

- many candidate predicates,
- many ambiguous entity types,
- many missing implicit arguments, and
- nontrivial uncertainty.

A typed SUMO backbone helps by providing:

- **type-guided disambiguation** (only a few predicate choices will type-check),
- **search control** (only inference paths consistent with domains/ranges are explored),
- **abductive structure** (SUMO axioms suggest what missing facts could explain an observation),
- **data fusion hooks** (dataset fields map into well-typed predicates).

4.2 PLN-friendly rule templates derived from SUMO

Many SUMO axioms compile into reusable inference templates.

```
; If x is an instance of A, and A is a subclass of B, then x is an instance of B.  
(: Instance (-> Entity Class Prop))
```

```
(rule  
  (and (Instance $x $A) (Subclass $A $B))  
  (Instance $x $B))
```

Domain/range enforcement as soft inference. Instead of treating domain/range violations as immediate rejection, you can treat them as evidence signals:

- If $R(a, b)$ is asserted but a is not typed as the domain class, infer either: (a) a type correction candidate for a , or (b) negative evidence against $R(a, b)$.

4.3 Combining evidence from text and data

Maintain multiple contexts and then combine them in a controlled way:

- **C:text:** semantic parses, subjective reports, policy documents.
- **C:data:** structured datasets, extracted graphs, measured signals.
- **C:model:** consequences derived from SUMO axioms + PLN inference.

You can implement aggregation rules such as:

- merge evidence when two assertions unify,
- promote conclusions when multiple independent sources agree,
- preserve conflicts explicitly rather than collapsing them.

5 Connecting SUMO to Hyperseed

5.1 What the SUMO $\leftrightarrow$ Hyperseed bridge should be

For practical use, the bridge should not be a brittle one-to-one mapping. It should be a *graded correspondence* that:

- can say “SUMO::Trust is close to HS::Trustworthiness” without claiming identity,
- is context-sensitive (a legal trust vs a cryptographic trust vs a social trust),
- composes (bridges can be chained), and
- can be used both as an *interpretation tool* and as *inference guidance*.

5.2 Representing mappings as graded correspondences

Define a bridge object:

```
(: Bridge Type)
(: mkBridge (-> Symbol Symbol Context TV Bridge))

; "SUMO symbol S corresponds to Hyperseed symbol H in context C with degree tv"
(: Corresponds (-> Symbol Symbol Context TV Prop))
```

Examples:

```
(: C:ethics Context)
(: C:crypto Context)
(: C:bio Context)

(Corresponds SUMO::Process HS::Process C:bio (pbit 0.90 0.05))
(Corresponds SUMO::Agreement HS::Communication C:ethics (pbit 0.70 0.20))
(Corresponds SUMO::Program HS::Program C:crypto (pbit 0.85 0.10))
```

5.3 Using the bridge for inference guidance

Given a SUMO-level assertion P, do:

1. Find Hyperseed concepts that correspond to the main predicates/classes in P.
2. Use Hyperseed-specific inference rules or heuristics (e.g. context shifts, pattern flow, value paraconsistency) to propose:
 - additional latent variables to consider,
 - likely missing facts,
 - plausible causal explanations,
 - which contradictions are “allowed tensions” vs “needs repair”.
3. Translate any resulting hypotheses back into SUMO-level candidates, then type-check and test them against data.

This makes Hyperseed a *semantic and methodological compass* rather than a replacement for SUMO.

6 Domain examples

6.1 AI ethics example: policy compliance and value conflicts

Scenario. Text: “Company Acme deployed facial recognition in public spaces without consent.”
Semantic parsing yields uncertain candidates (illustrative):

```
(Assert C:text
  (SUMO::agent DOM::DeployAction Acme)
  (stv 0.8 0.5))

(Assert C:text
  (SUMO::patient DOM::DeployAction DOM::FacialRecognitionSystem)
  (stv 0.7 0.5))

(Assert C:text
  (DOM::inContext DOM::FacialRecognitionSystem DOM::PublicSpace)
  (stv 0.7 0.4))

(Assert C:text
  (DOM::hasConsent DOM::DeployAction DOM::NoConsent)
  (stv 0.6 0.4))
```

SUMO guidance. Type constraints can quickly rule out mis-parses, e.g. `agent` expects an Entity of an appropriate type; `patient` expects an object.

Hyperseed guidance. Map the situation into Hyperseed ethical notions and allow paraconsistent values:

- one can have evidence both for “improves safety” and “violates privacy”;
- represent this as a structured, non-explosive tension rather than forcing collapse.

Illustrative Hyperseed-oriented derived atoms:

```
; These are not claimed SUMO facts; they are HS-framed interpretations.
(Assert C:model
  (HS::ValueImpact DOM::DeployAction HS::Safety)
  (pbit 0.60 0.20))

(Assert C:model
  (HS::ValueImpact DOM::DeployAction HS::Autonomy)
  (pbit 0.15 0.70))

(Assert C:model
  (HS::ValueImpact DOM::DeployAction HS::Compassion)
  (pbit 0.10 0.65))
```

Then reasoning can seek policies or actions that satisfy constraints under conflict, rather than expecting a single scalar utility.

6.2 Crypto-finance example: on-chain pattern recognition + causal explanations

Assumptions. We assume: (i) access to crypto datasets (transactions, addresses, contract events), (ii) tools that recognize patterns (wash trading, mixers, sandwich attacks, etc.).

Data ingestion into SUMO. Represent a transaction and its roles:

```
(: Tx123 Entity)
(: Alice Entity)
(: Bob Entity)
(: TokenX Entity)
```

```
(Assert C:data (DOM::isTransaction Tx123) (stv 0.99 0.95))
(Assert C:data (DOM::sender Tx123 Alice) (stv 0.99 0.95))
(Assert C:data (DOM::receiver Tx123 Bob) (stv 0.99 0.95))
(Assert C:data (DOM::asset Tx123 TokenX) (stv 0.99 0.95))
```

```
(Assert C:data (DOM::pattern DOM::WashTradePattern Tx123) (stv 0.85 0.70))
```

Bridge into Hyperseed. Crypto-finance often benefits from Hyperseed notions like:

- engineered artifacts: programs, protocols, contracts,
- social trust and coordination (communities, adversaries, incentives),
- pattern flow networks (how behaviors propagate across agents and time),
- approximate morphisms (patterns transferring across chains or market regimes).

A small example of explanatory inference:

- If the pattern tool suggests wash trading,
- and SUMO + domain rules say wash trading is a kind of market manipulation,
- then infer a candidate causal explanation: “a coordinated group executed trades to create a misleading price signal.”

Store the explanation with explicit uncertainty and keep it revisable.

6.3 Biology example: metabolic pathways as pattern-flow networks

Assumptions. We assume an integrated AtomSpace of bio-ontologies (pathways, enzymes, metabolites, GO terms, etc.), and we want to explain pathways and infer missing causal structure.

Typed pathway ingestion. Represent a reaction:

```
(: R1 Entity)
(: Glucose Entity)
(: G6P Entity)
(: Hexokinase Entity)

(Assert C:data (DOM::isReaction R1) (stv 0.99 0.95))
(Assert C:data (DOM::substrate R1 Glucose) (stv 0.99 0.95))
(Assert C:data (DOM::product R1 G6P) (stv 0.99 0.95))
(Assert C:data (DOM::enzyme R1 Hexokinase) (stv 0.95 0.85))
```

SUMO guidance. SUMO can supply generic structure: **Process**, **agent**, **patient**, subProcess relations, etc. The type checker prevents category mistakes (enzyme vs metabolite vs process).

Hyperseed guidance: causal explanation as a graded causal attribution. A convenient bridge is to interpret “enzyme + substrate produces product” using a graded causal triple schema:

```
; Gamma(a,b,c) ~ "interaction of a and b caused c" with a degree.
(Assert C:model (HS::Gamma Hexokinase Glucose G6P) (stv 0.80 0.60))
```

This lets you:

- infer missing enzymes or intermediates as abductive hypotheses,
- compare alternative explanations by simplicity/weakness criteria,
- propagate pathway explanations across contexts (tissue, condition, organism).

6.4 Biology example: causal explanations for genomics data

Goal. Given gene expression / genomic signals, infer plausible causal structures that explain observed phenotypes while avoiding overfitting.

Mechanism.

1. Convert observations into evidence-bearing propositions in context **C:data**.
2. Use bio-ontology links to suggest candidate mechanisms (pathway membership, regulatory relations, protein interactions).
3. Use SUMO typing to avoid nonsensical hypotheses (e.g. treating a gene as an action).
4. Use Hyperseed-style guidance: favor explanations that do not make unnecessary distinctions, and treat conflicts as explicit rather than exploding.

A minimal schematic objective for selecting among candidate causal graphs G :

$$\text{score}(G) = \underbrace{\text{fit}(G; \text{data})}_{\text{predictive/explanatory}} - \lambda \cdot \underbrace{\text{complexity}(G)}_{\text{unnecessary distinctions}} + \mu \cdot \underbrace{\text{transfer}(G)}_{\text{cross-context reuse}} .$$

This is not a single algorithm; it is a way to keep “explain” and “generalize” as first-class, checkable pressures in the inference loop.

7 Suggested MVP experiments

A practical first iteration can be small but meaningful:

- Step 1:** Pick a *restricted SUMO slice* (a few hundred classes/relations) that cover: agents, actions, processes, artifacts, communication, and a small ethics vocabulary.
- Step 2:** Auto-generate MeTTa symbol declarations and relation signatures from SUMO domain/range axioms; run the type-check passes.
- Step 3:** Build a semantic-parse adapter that emits `Assert(C:text, P, tv)` atoms.
- Step 4:** Add a dataset adapter:
- AI ethics: policy corpus + compliance annotations,
 - crypto: transaction/address graph + pattern tool outputs,
 - bio: pathway graphs + expression matrices.
- Step 5:** Implement a small bridge table: `Corresponds(SUMO::X, HS::Y, C, tv)` for 30–80 key concepts.
- Step 6:** Evaluate:
- type-checking catches mis-parses and KB inconsistencies,
 - SUMO-guided inference improves precision/recall vs untyped inference,
 - Hyperseed-guided inference improves explanation quality and transfer (e.g. fewer brittle heuristics, more reusable causal schemata).

8 Summary

Metta-stasizing SUMO in a way that supports formal type-checking, PLN-style inference, and Hyperseed alignment is feasible if you:

- make the type system explicit (Class, RelSym, signatures, subtyping),
- treat evidence and context as first-class (not bolted on),
- keep both direct and reified predicate forms (fast object reasoning vs meta reasoning),
- treat the SUMO $\leftrightarrow$ Hyperseed link as graded, contextual correspondence,
- use the bridge primarily to guide inference and explanations, not to demand identity.

The result is a pipeline where semantic parses and datasets become well-typed SUMO-level assertions, and Hyperseed becomes an inference compass that helps connect everyday problems to deeper explanatory patterns.